HARI 4

Cloud: Observability, Deployment, dan CI/CD

Melacak satu transfer lintas service, mengemas service ke *container*, menjalankannya di Kubernetes, lalu merilisnya melalui *pipeline* CI/CD

DISIAPKAN UNTUK

Peserta In-house Training

Endy Muhardin

PT ArtiVisi Intermedia · 29 Oktober 2026

HARI 4

Agenda hari ini

Waktu	Topik	Format
08:00 – 08:15	Rekap Hari 3 dan penyamaan posisi dari *checkpoint* `lab-07` bagi peserta yang tertinggal	Diskusi
08:15 – 09:00	Peta *concern* microservices. Lanskap *cloud* untuk perbankan: model *deployment*, *managed* vs *self-managed*, service pendukung, analisis *build*/*reuse*/*buy*, regulasi, posisi Kubernetes	Presentasi + diskusi
09:00 – 10:15	**Lab 8**: melacak satu transfer lintas tiga service di Grafana	*Hands-on*
10:30 – 12:00	*Containerization*, konfigurasi, Kubernetes, dan **Lab 9** (1): seluruh sistem berjalan di k3d	Demo + *hands-on*
12:00 – 13:00	Istirahat makan siang	—
13:00 – 13:30	**Lab 9** (2): *rolling update*, perubahan konfigurasi, *probe*	*Hands-on*
13:30 – 14:15	CI/CD per service dan pemetaannya ke Jenkins dan GitHub Actions; GitOps sekilas	Presentasi + demo
14:15 – 15:45	**Lab 10**: dari *commit* hingga *rolling update*	*Hands-on*
16:00 – 17:00	Operasional: HPA, penempatan *observability stack*, OpenShift. Peta *concern* lengkap, ringkasan keputusan desain, diskusi dan tanya jawab	Presentasi + diskusi

PETA CONCERN MICROSERVICES

Concern yang dibahas di Hari 4

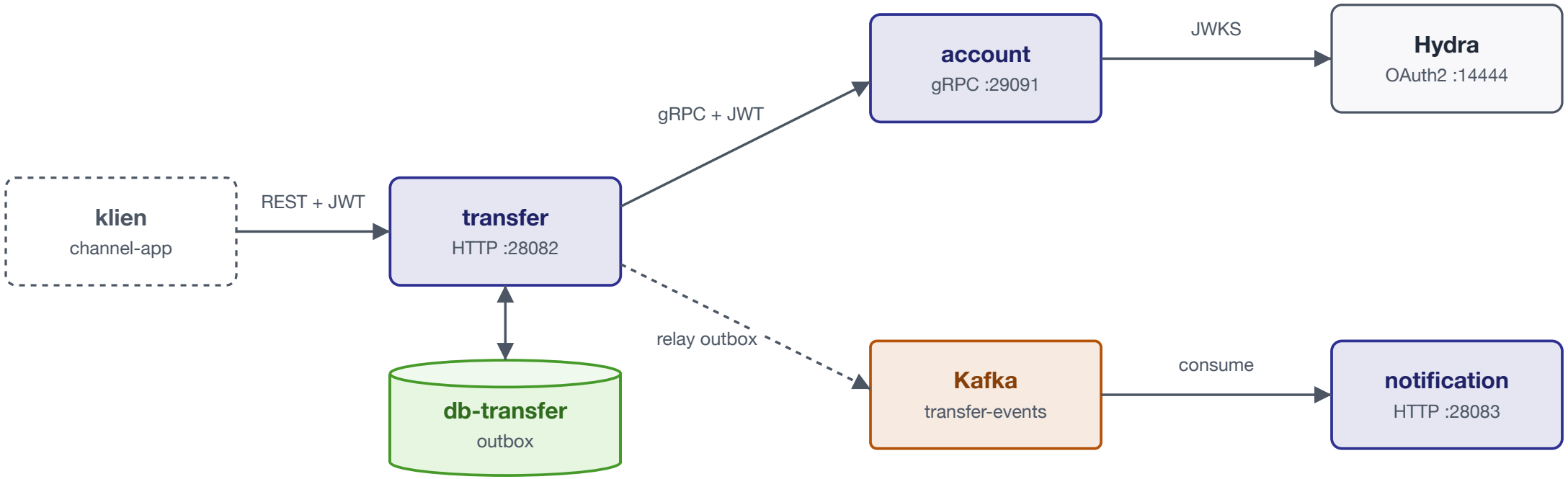
#	*Concern*	Pertanyaan yang dijawab	Hari dan lab	Di kode (*library*)	Di platform atau infrastruktur
1	*Decomposition* dan kepemilikan data	*Service boundary* dan pemilik data	H1, Lab 1–2	—	*Database per service*
2	Komunikasi *synchronous* dan *asynchronous*	REST, gRPC, *messaging*	H1–H3, Lab 2, 4, 5	`net/http`, grpc-go, kafka-go	Kafka
3	*Resiliency*	Tetap berfungsi saat dependensi lambat atau mati	H2, Lab 3–4	failsafe-go	*Probe*, *restart*, *rolling update* (H4)
4	Konsistensi data	Perubahan data lintas database	H2–H3, Lab 5–6	Saga, *outbox*, *idempotency*	—
5	*Security*	Identitas pengguna (SSO) dan service, serta hak akses	H3, Lab 7	JWT *middleware* dan *interceptor*	*Authorization server* atau IdP (Hydra di lab, Keycloak), API *gateway*
6	***Observability***	Kondisi tiap service dan integrasi antarservice	H3–H4, Lab 8	`slog`, Prometheus *client*, OTel SDK	Grafana, Loki, Tempo, Prometheus
7	***Configuration* (termasuk *centralized config* dan *secret*)**	Nilai yang berbeda per *environment*	H1, Lab 1 (*env var*); H4, Lab 9 (ConfigMap, Secret)	`LoadConfig`	ConfigMap, Secret, *secret manager*
8	***Service discovery* dan *load balancing***	Alamat service tujuan	H4, Lab 9	— (alamat dari konfigurasi)	Kubernetes Service (DNS), Ingress
9	***Packaging* dan *deployment***	Cara service dijalankan dan diperbarui	H4, Lab 9	Dockerfile	Kubernetes Deployment, *probe*, *rolling update*
10	***Automation* (CI/CD)**	Dari *commit* ke *production*	H4, Lab 10	Target `Makefile`	*Pipeline* (git server dan runner CI, Jenkins)

CATATAN

Hari 1–3 banyak membahas sisi kode. Hari 4 membahas *concern* yang sebagian besar ditangani platform: Kubernetes dan *pipeline*. *Observability* hanya muncul di Lab 8, yang mempraktikkan konsep dan kode dari akhir Hari 3.

STUDI KASUS

Posisi sistem di awal Hari 4



- Hasil Lab 1–7: saga dengan kompensasi, *outbox*, *idempotent consumer*, gRPC dengan *circuit breaker*, dan JWT dari Hydra.
- Hari ini sistem yang sama diberi instrumentasi, dikemas ke *container*, dijalankan di Kubernetes, lalu dirilis melalui *pipeline*.

01

Lanskap *cloud* untuk perbankan

Model *deployment*, *managed* vs *self-managed*, service pendukung, *build/reuse/buy*, regulasi, dan posisi Kubernetes

Empat model *deployment*

Model	Ciri	Kelebihan	Pertimbangan
On-premise	Server milik sendiri di *data center* sendiri	Kendali penuh atas data dan jaringan	Kapasitas harus dibeli di muka, dan tim mengelola sendiri seluruh *stack*
Private cloud	Platform *cloud* khusus satu organisasi	Kendali tetap tinggi, penyediaan *resource* lebih cepat	Investasi platform dan tim operasional
Public cloud	*Resource* milik *cloud provider*, digunakan bersama oleh banyak pelanggan	Elastis, pilihan *managed service* lengkap	Klasifikasi data dan lokasi penyimpanan harus sesuai regulasi
Hybrid	Gabungan model di atas	Penempatan dipilih per *workload*	Konektivitas, identitas, dan *observability* lintas lokasi

CATATAN

Pilihan model ditentukan per *workload*, berdasarkan klasifikasi data yang diolah *workload* tersebut.

MODEL DEPLOYMENT

Managed vs self-managed

Managed service

- *Cloud provider* mengurus instalasi, *patch*, *backup*, dan *high availability*
- Tim fokus pada aplikasi
- Konfigurasi terbatas pada pilihan yang disediakan *cloud provider*
- Contoh: database *managed*, Kafka *managed*, Kubernetes *managed*

Self-managed

- Tim menjalankan sendiri PostgreSQL, Kafka, Kubernetes
- Seluruh parameter bisa diatur
- Beban operasional: *upgrade*, *monitoring*, *restore* data, rotasi sertifikat
- Lazim di *on-premise* dan *private cloud*

TIPS

Untuk memilih, tanyakan siapa yang dihubungi bila service tersebut mati pukul 02.00, dan apakah pihak tersebut mampu mengaktifkannya kembali.

SERVICE PENDUKUNG

Service pendukung tersedia untuk semua bahasa

Kebutuhan	Produk (tidak terikat bahasa)	Integrasi di Java (Spring)	Integrasi di Go (studi kasus)
Service registry dan *discovery*	Consul, etcd, Kubernetes Service	Spring Cloud Netflix (Eureka), Spring Cloud Consul	Kubernetes Service DNS, alamat dari `ACCOUNT_GRPC_ADDR`
Config server	Spring Cloud Config Server, Consul KV, ConfigMap	Spring Cloud Config *client*	*Environment variable* dari ConfigMap, dibaca `LoadConfig`
Secret	Vault, *secret manager* milik *cloud provider*	Spring Cloud Vault	Secret Kubernetes, diisi dari Vault oleh External Secrets Operator tanpa perubahan kode
SSO dan IdP	Keycloak, Ory Hydra, ZITADEL, IdP komersial	Spring Security: *resource server*, OAuth2 *client*, dan *Authorization Server embedded*	golang-jwt dan keyfunc (JWKS), `x/oauth2/clientcredentials`
Log server	Loki, Elasticsearch, OpenSearch	*Appender* Logback atau Log4j2, *agent* OpenTelemetry Java	`slog` JSON ke *stdout* dan *bridge* `otelslog`
Monitoring dan *tracing*	Prometheus, Grafana, OTel Collector, Tempo, Jaeger	Micrometer (Actuator), *agent* OpenTelemetry Java	`prometheus/client_golang`, OTel Go SDK, `otelhttp`, `otelgrpc`

CATATAN

Produk di kolom kedua dapat digunakan oleh service dari bahasa apa pun. Yang berbeda adalah integrasi di sisi service. Spring Cloud menyediakan integrasi siap pakai, sedangkan service Go di studi kasus bersandar pada Kubernetes, OTel SDK, dan beberapa *library* kecil. Perbandingan bahasa dan *library* per *concern* (Go, Java, Node.js, PHP) ada di Hari 1. Consul dan Vault menggunakan lisensi BSL sejak 2023, sedangkan OpenBao adalah *fork* Vault berlisensi MPL.

Analisis *build*, *reuse*, dan *buy* per *concern*

Concern	*Build* (tulis sendiri)	*Reuse* (*open source*, dioperasikan sendiri)	*Buy* (*managed service* atau produk komersial)	Pertimbangan untuk bank
Konfigurasi dan *secret*	`LoadConfig` dengan validasi saat *startup*	ConfigMap, OpenBao, External Secrets Operator	*Secret manager* milik *cloud provider*, Vault (lisensi BSL, edisi Enterprise atau HCP)	Lokasi penyimpanan *credential*, audit akses, rotasi
IdP dan SSO	*Middleware* dan *interceptor* JWT, atau *authorization server embedded* (Spring Security, zitadel/oidc)	Keycloak, Ory Hydra	IdP komersial atau *managed*	Semua service bergantung pada IdP, sehingga tim harus sanggup menjaga ketersediaannya
Observability	Setup telemetri, *handler* `slog`	Prometheus, Grafana, Loki, Tempo, OTel Collector	Platform *observability* SaaS	*Log* dan *trace* bisa berisi data nasabah. Lokasi data dan retensi menentukan boleh tidaknya SaaS digunakan
Messaging	—	Apache Kafka	Kafka *managed*, distribusi Kafka komersial	Kemampuan tim melakukan *upgrade*, menambah *broker*, dan memulihkan data
Container platform	—	Kubernetes, k3s	OpenShift, Kubernetes *managed*	Biaya lisensi dibandingkan biaya tim platform, dukungan vendor saat audit
CI/CD	Target `make`, *pipeline* per service	Jenkins, git server + runner	Layanan CI SaaS	Akses runner ke jaringan *production*, penyimpanan *credential* untuk *deploy*

TIPS

Tulis sendiri hanya *glue code* yang tipis: pembaca konfigurasi, *middleware*, dan setup telemetri. Perangkat lunak infrastruktur cukup di-*reuse*. Pilih *buy* ketika kapasitas operasional tim atau ketentuan regulasi lebih cocok dengan produk yang didukung vendor.

REGULASI

Regulasi dan klasifikasi data

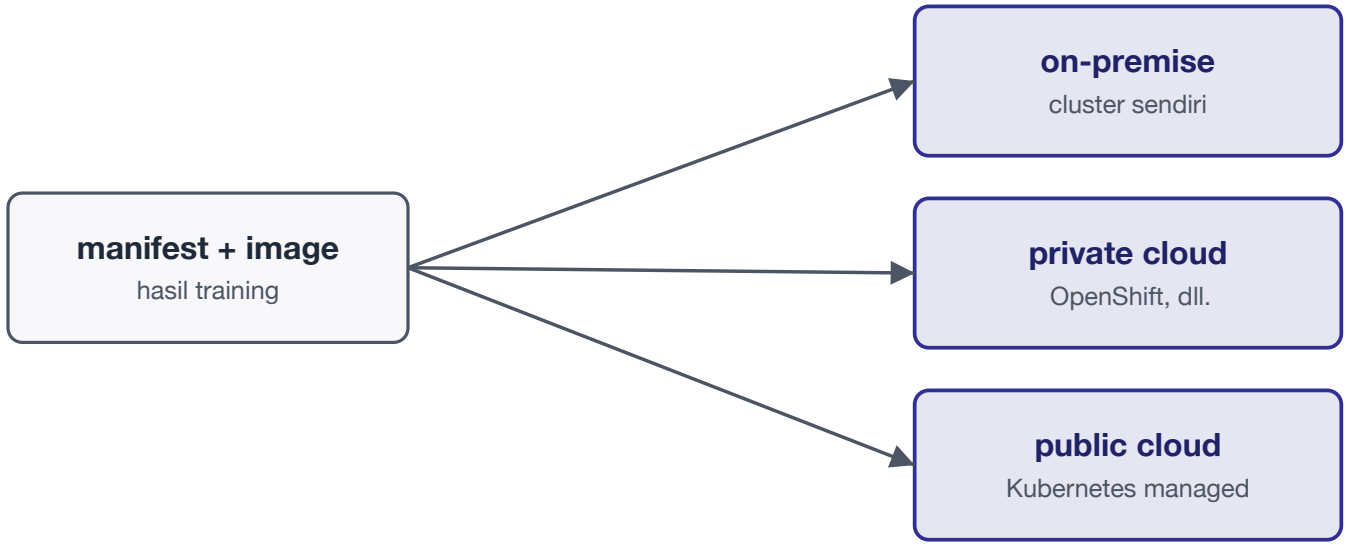
- Penempatan sistem elektronik, klasifikasi data, dan manajemen risiko TI di sektor perbankan diatur ketentuan OJK.
- Data nasabah dan data transaksi umumnya masuk kelas paling ketat, sehingga *workload* yang mengolahnya cenderung ditempatkan di *on-premise* atau *private cloud*.
- *Workload* dengan data yang tidak sensitif, misalnya konten publik atau *environment* pengembangan, lebih leluasa ditempatkan di *public cloud*.
- Keputusan penempatan perlu dicatat beserta alasannya, karena auditor akan menanyakan dasar keputusan tersebut.

PERHATIAN

Ketentuan regulasi berubah dari waktu ke waktu. Rujuk ketentuan yang berlaku dan kebijakan internal organisasi sebelum memindahkan *workload*.

PORTABILITAS

Manifest Kubernetes yang sama berjalan di semua model *deployment*



- Objek yang sama di semua tempat: Deployment, Service, ConfigMap, Secret
- Perbedaan antar-*cluster* ditampung di *overlay* Kustomize
- Saat kebijakan penempatan berubah, aplikasi tidak perlu ditulis ulang

CATATAN

Portabilitas berlaku selama aplikasi hanya bergantung pada API Kubernetes standar. Fitur khusus satu *cloud provider*, misalnya *load balancer* atau *storage class* tertentu, perlu dipisahkan ke *overlay*.

02

Lab 8: Observability lintas service

Melacak satu transfer lintas tiga service di Grafana

LAB 8

Melacak satu transfer di Grafana

Tujuan

- Menemukan service yang lambat langsung dari data *trace*
- Membaca status *circuit breaker* dari *dashboard*
- Berangkat dari lonjakan di metrik, lalu menghubungkan *trace* dan *log* melalui `trace_id`

Yang dikerjakan

- Instrumentasi `slog`, Prometheus, dan OpenTelemetry di tiga service
- Menjalankan `otel-lgtm` dari `compose.yaml`
- Memasang *fault delay* di account dengan `make fault-slow`, lalu mencari *span* yang lambat di Tempo
- Mengamati *breaker* berubah ke *open* di *dashboard*

TIPS

Langkah lengkap ada di **Panduan Lab, bab Lab 8**. Titik awal: *tag* `lab-07`. Peserta yang tertinggal cukup melakukan `git checkout` ke *tag* tersebut.

CATATAN

Konsep, kode instrumentasi, dan `otel-lgtm` dibahas di akhir Hari 3, sesi *Observability* lintas service.

03

Containerization

Multi-stage build, *static binary* Go, *image* minimal, dan rekap 12-factor dari Hari 1

KEPUTUSAN DESAIN

Container image sebagai artefak rilis

Kebutuhan

- Artefak yang sama berjalan di laptop, CI, dan *production*
- Dependensi *runtime* ikut terkemas, sehingga tidak ada perbedaan versi antar-*server*
- Isolasi antar-service yang berjalan di *host* yang sama

Opsi yang dipertimbangkan

- *Binary* langsung di VM, dijalankan systemd
- Paket OS (deb, rpm)
- *Container image* (format OCI)

Pilihan di studi kasus

- Satu *image* per service per *commit*, dibangun dengan *multi-stage build*
- `FROM scratch`, *non-root* (`USER 65532:65532`), konfigurasi dari *environment variable*

Konsekuensi dan *trade-off*

- Membutuhkan *container runtime*, *container registry*, dan pemindaian *image*
- *Image* harus dibangun ulang ketika Go atau sertifikat CA mendapat *patch*
- `scratch` tidak memiliki *shell*. Pemeriksaan di dalam *pod* dilakukan dengan `kubectl debug` (*ephemeral container*)

Static binary Go menghasilkan *image* yang kecil

- Dengan `CGO_ENABLED=0`, hasil *build* Go berupa satu *static binary* yang tidak membutuhkan *library* sistem.
- *Image* akhir cukup berisi *binary* tersebut dan sertifikat CA, berukuran belasan hingga puluhan megabyte.
- Tanpa *shell* dan *package manager* di *image*, *attack surface* lebih kecil dan hasil *scanning* lebih bersih.

Ukuran tipikal

Base image	Ukuran kira-kira
`golang:1.27.2-alpine` (*builder*)	± 300 MB
`alpine` + *binary*	± 30 MB
`scratch` + *binary*	± 22 MB
`scratch` + *binary*, terkompresi	± 8 MB

Studi kasus: *binary* 21–23 MB per service. `docker images` dengan *containerd image store* menampilkan `CONTENT SIZE` ± 8 MB dan `DISK USAGE` ± 32 MB.

Multi-stage build untuk service transfer

```
FROM golang:1.27.2-alpine AS build
ENV GOWORK=off CGO_ENABLED=0
WORKDIR /src
COPY proto/go.mod proto/go.sum proto/
COPY transfer/go.mod transfer/go.sum transfer/
WORKDIR /src/transfer
RUN go mod download
COPY proto/ /src/proto/
COPY transfer/ /src/transfer/
RUN go build -trimpath -ldflags "-s -w" \
      -o /out/transfer .

FROM scratch
COPY --from=build /etc/ssl/certs/ca-certificates.crt \
     /etc/ssl/certs/
COPY --from=build /out/transfer /transfer
USER 65532:65532
EXPOSE 8082
ENTRYPOINT ["/transfer"]
```

Sumber: `transfer/Dockerfile` · *tag* `lab-09`

- *Stage* pertama: kompilasi dengan *toolchain* lengkap
- *Stage* kedua: `FROM scratch`, hanya menyalin hasil *build*
- `-trimpath -ldflags "-s -w"`: *path* lokal dan simbol *debug* dibuang
- `GOWORK=off`: *build* hanya melibatkan modul service dan `proto`, sehingga perubahan service lain tidak membuat *cache* menjadi tidak valid
- `go.mod` disalin terlebih dahulu agar *layer* hasil `go mod download` tetap di-*cache*
- *Build context* `studi-kasus/` karena account dan transfer membutuhkan modul `proto` (`replace ../proto`). Notification tidak menyalin `proto/`

KEAMANAN IMAGE

Non-root dan UID acak

`USER 65532:65532`

- Proses tidak berjalan sebagai `root` walaupun *image* dijalankan tanpa opsi tambahan
- UID numerik dibutuhkan agar Kubernetes bisa memverifikasi `runAsNonRoot`
- `scratch` tidak memiliki `/etc/passwd`, sehingga nama pengguna tidak bisa digunakan

UID acak

- OpenShift dengan SCC `restricted-v2` menjalankan *container* dengan UID acak dari rentang milik *namespace*
- *Binary* harus tetap berjalan dengan UID berapa pun
- Service tidak menulis ke *filesystem*: *log* ke *stdout*, data ke database

TIPS

Uji sendiri: `docker run --user 1000770000:0 ...` harus berjalan normal. Bila gagal, kemungkinan besar service mencoba menulis *file* ke direktori milik UID lain.

REKAP 12-FACTOR

Prinsip 12-factor dari Hari 1 yang sudah diterapkan di lab

Prinsip	Penerapan di studi kasus	Lab
II. *Dependencies*	Dependensi dideklarasikan di `go.mod` setiap modul dan ikut terkompilasi ke *binary*	Lab 1
III. *Config*	Seluruh konfigurasi dibaca dari *environment variable* oleh satu `LoadConfig`, tanpa nilai *default*	Lab 1, 9
IV. *Backing services*	Database, Kafka, Hydra, dan *collector* diakses lewat URL dari konfigurasi, sehingga bisa diganti tanpa *build* ulang	Lab 1–2, 5, 7–8
V. *Build, release, run*	Setiap *commit* menghasilkan satu *image*. Konfigurasi ditambahkan saat *deploy* melalui ConfigMap dan Secret	Lab 9–10
VI. *Processes*	Service tidak menyimpan *state* di memori karena *state* disimpan di database	Lab 1, 9
VII. *Port binding*	Service membuka *port* HTTP dan gRPC sendiri: `HTTP_ADDR`, `GRPC_ADDR`	Lab 1, 4
VIII. *Concurrency*	Kapasitas ditambah dengan menambah *replica* (`kubectl scale`) dan tidak memperbesar satu proses	Lab 9
IX. *Disposability*	*Startup* cepat dan *graceful shutdown* saat menerima `SIGTERM`, ditambah `preStop` di Kubernetes	Lab 1, 9–10
X. *Dev/prod parity*	*Image* dan manifest *base* yang sama di k3d dan di *server* pelatihan. Perbedaannya hanya di *overlay*	Lab 9–10
XI. *Logs*	Service menulis *log* ke *stdout* (format JSON sejak Lab 8), lalu platform yang mengumpulkannya	Lab 1, 8

Tabel lengkap ada di Hari 1 (anatomi service). Faktor I (*Codebase*: satu *module* dan satu *pipeline* per service) dan XII (*Admin processes*: skema database diterapkan saat *startup*) tidak diulang di sini.

KONFIGURASI

Startup dihentikan bila ada konfigurasi wajib yang belum diisi

```go
func LoadConfig() (Config, error) {
    var missing []string
    get := func(name string) string {
        v := os.Getenv(name)
        if v == "" {
            missing = append(missing, name)
        }
        return v
    }
    cfg := Config{
        HTTPAddr:        get("HTTP_ADDR"),
        DatabaseURL:     get("DATABASE_URL"),
        AccountGRPCAddr: get("ACCOUNT_GRPC_ADDR"),
        // ... Kafka, OAuth, OTel
    }
    if len(missing) > 0 {
        return Config{}, fmt.Errorf(
            "missing required environment variables: %s",
            strings.Join(missing, ", "))
    }
    if err := checkEndpointURL(
        "OTEL_EXPORTER_OTLP_ENDPOINT",
        cfg.OTelEndpoint); err != nil {
        return Config{}, err // juga bila port kosong
    }
    return cfg, nil
}
```

Sumber: `transfer/config.go` · *tag* `lab-08`

- *Environment variable* yang kosong menghentikan *startup* dengan pesan *error* yang menyebut nama variabel tersebut
- *Pod* dengan ConfigMap yang salah langsung gagal dan terlihat di `kubectl get pods`
- Nilai *default* yang digunakan tanpa terdeteksi, misalnya `localhost`, baru ketahuan saat transaksi pertama gagal
- Nilai yang formatnya salah juga ditolak saat *startup*. `OTEL_EXPORTER_OTLP_ENDPOINT` tanpa *port* membuat gRPC menghubungi *port* 443, dan semua ekspor gagal tanpa pesan *error* setelah service berjalan

04

Deployment ke Kubernetes

Deployment, Service, ConfigMap/Secret, konfigurasi terpusat, *probe*, dan *rolling update* di k3d

KEPUTUSAN DESAIN

Kubernetes sebagai platform *deployment*

Kebutuhan

- Banyak service, masing-masing dengan beberapa *replica*
- Penjadwalan ke *node*, *restart* otomatis, *rolling update*
- *Service discovery* dan konfigurasi per *environment*

Opsi yang dipertimbangkan

- Docker Compose di beberapa VM
- HashiCorp Nomad
- Kubernetes: *vanilla*, OpenShift, atau Kubernetes *managed* milik *cloud provider*

Pilihan di studi kasus

- Manifest standar: Deployment, Service, ConfigMap, Secret, Ingress, disusun dengan Kustomize
- Deklaratif: manifest berisi *desired state*, *controller* menyamakan isi *cluster* dengan manifest tersebut (*self-healing*)
- Service DNS menggantikan *service registry*, dan manifest yang sama berjalan di OpenShift dan Kubernetes *managed*

Konsekuensi dan *trade-off*

- Platform yang kompleks: *upgrade cluster*, RBAC, jaringan, dan penyimpanan membutuhkan tim platform
- Berlebihan untuk satu atau dua service
- Fitur khusus distribusi tetap perlu dipelajari, misalnya Route dan SCC di OpenShift

KEPUTUSAN DESAIN

k3d sebagai *cluster* Kubernetes untuk training

Kebutuhan

- *Cluster* di laptop setiap peserta, mudah dibuat dan dihapus
- Kebutuhan RAM kecil, berjalan di Docker yang sudah terpasang
- *Image* hasil *build* lokal bisa langsung dijalankan tanpa *registry*

Opsi yang dipertimbangkan

- minikube
- kind (Kubernetes di dalam Docker)
- Kubernetes bawaan Docker Desktop, yang tidak tersedia di Docker Engine Linux
- k3d (k3s di dalam Docker)

Pilihan di studi kasus

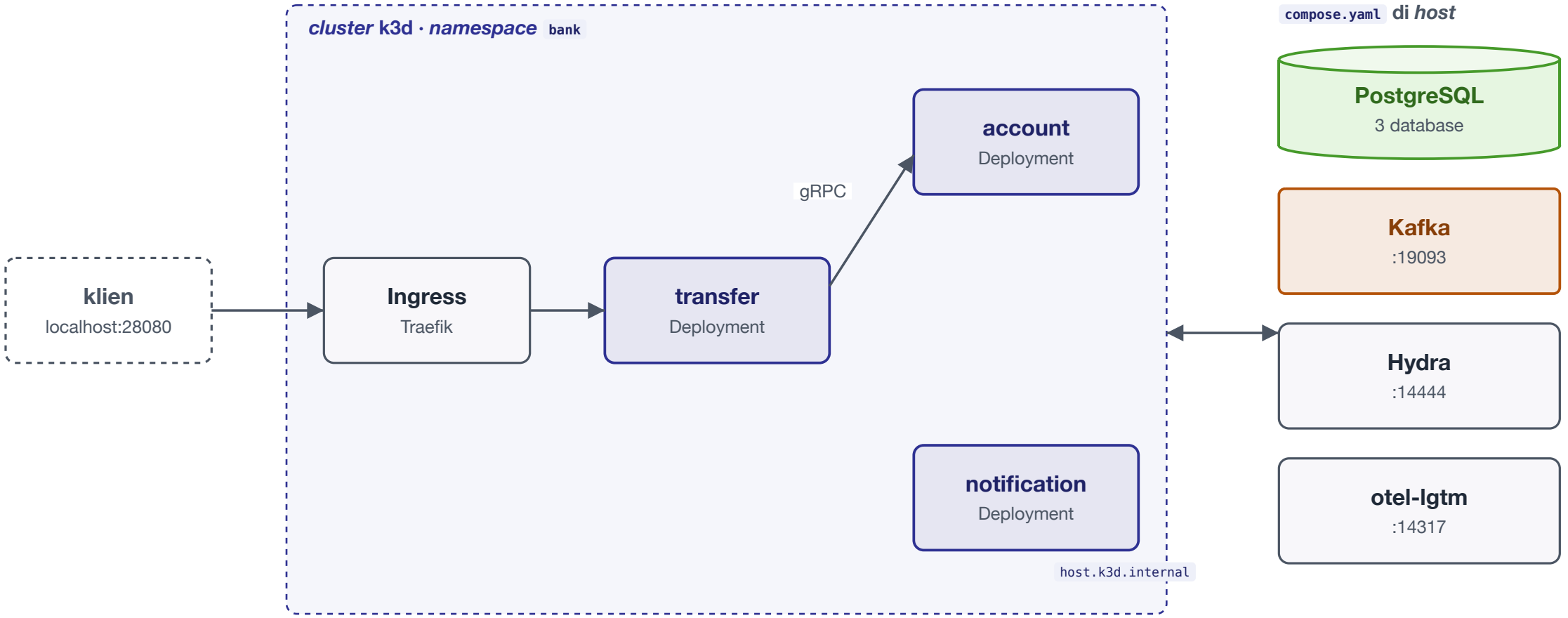
- k3d dengan versi k3s yang di-*pin* (`rancher/k3s:v1.36.5-k3s1`), dibuat dengan `make k3d-up` dan dihapus dengan `make k3d-down`
- Traefik Ingress bawaan k3s. *Port* 28080 laptop diteruskan oleh *load balancer* k3d ke Traefik (`-p 28080:80@loadbalancer`)
- `k3d image import` menyalin *image* dari Docker lokal ke *cluster*

Konsekuensi dan *trade-off*

- k3s berbeda dengan OpenShift: SCC dan Route dibahas di bagian OpenShift
- Komponen bawaan k3s (Traefik, metrics-server) belum tentu ada di *cluster* lain
- Akses ke infrastruktur lewat `host.k3d.internal` hanya berlaku di lab laptop

LAB 9

Arsitektur *deployment* di laptop



Objek Kubernetes di Lab 9

Objek	Fungsi	Di studi kasus
Deployment	Menjaga jumlah *replica* dan mengatur *rolling update*	Satu per service
Service	Nama DNS dan *load balancing* ke *pod* yang *ready*	`account:9091` dipanggil transfer lewat gRPC
ConfigMap	Konfigurasi yang tidak rahasia	`HTTP_ADDR`, `KAFKA_TOPIC`, `OAUTH_ISSUER`
Secret	Konfigurasi rahasia	`DATABASE_URL`, `OAUTH_CLIENT_SECRET`
Ingress	*Routing* HTTP dari luar *cluster*	`/transfers` lewat Traefik bawaan k3s
Kustomize	*Base* bersama, *overlay* per *cluster*	`k8s/base`, `k8s/overlays/k3d`

CATATAN

Manifest sudah disediakan di repositori studi kasus. Fokus Lab 9 adalah menerapkan, membaca, lalu memodifikasi manifest tersebut.

KONFIGURASI TERPUSAT

Pilihan tempat menyimpan konfigurasi

Opsi	Cara kerja	Di studi kasus	Layak digunakan ketika
Environment variable	Diisi oleh proses yang menjalankan service: Makefile, Docker Compose, atau Kubernetes	Lab 1–10	Selalu, sebagai antarmuka antara service dan platform
ConfigMap dan Secret	Objek Kubernetes, diteruskan ke *container* sebagai *environment variable* atau *file*	Lab 9–10	Service berjalan di Kubernetes dan nilainya berbeda per *cluster*
Config server (Spring Cloud Config, Consul KV)	Service mengambil nilai dari server pusat saat *startup*. Nilai baru dapat dimuat tanpa *restart* melalui *refresh* (Spring Cloud Config) atau *watch* (Consul)	Tidak digunakan	Banyak service berbagi nilai yang sering berubah, atau nilai harus berubah tanpa *restart*
Secret manager (Vault, *secret manager* milik *cloud provider*)	Penyimpanan terenkripsi dengan audit akses dan rotasi *credential*	Tidak digunakan	*Credential production* dengan kewajiban audit dan rotasi
External Secrets Operator	*Operator* di *cluster* menyalin nilai dari *secret manager* ke Secret Kubernetes	Tidak digunakan	Sumber nilai di *secret manager*, sementara service tetap membaca *environment variable*

PERHATIAN

Secret Kubernetes disimpan di etcd dalam bentuk base64, tidak terenkripsi kecuali *encryption at rest* diaktifkan di *cluster*. RBAC menentukan siapa yang boleh membacanya. Secret *overlay* k3d di repositori hanya berisi *credential* lab.

Konfigurasi statis dan konfigurasi dinamis

Statis: dibaca sekali saat *startup* (studi kasus)

- `LoadConfig` membaca *environment variable* satu kali. Perubahan ConfigMap tidak terlihat oleh *pod* yang sedang berjalan
- Nilai baru berlaku setelah `kubectl rollout restart`, yang berjalan sebagai *rolling update* dengan *readiness probe* seperti rilis kode
- Nilai yang digunakan setiap *pod* jelas, yaitu nilai saat *pod* tersebut dibuat
- Lab 9: setelah `KAFKA_GROUP_ID` dihapus dan `rollout restart`, *pod* baru gagal *start* dan *rollout* tertahan, sementara *pod* lama tetap melayani

Dinamis: dimuat ulang tanpa *restart*

- Service memantau sumber konfigurasi: *config server*, *watch* Consul, atau *file* dari ConfigMap yang di-*mount* sebagai *volume*
- Kode harus memvalidasi nilai baru dan menangani perubahan nilai di tengah *request* yang sedang berjalan
- Saat menyelidiki gangguan, nilai yang berlaku pada waktu kejadian lebih sulit dipastikan
- Cocok untuk *feature flag* dan batas operasional, misalnya *rate limit*

CATATAN

Aturan tanpa nilai *default* berlaku untuk semua opsi. *Key* yang hilang di sumber mana pun menghentikan *startup* dengan pesan *error*. Service tidak berjalan dengan nilai tebakan.

PROBE

Readiness dan *liveness probe* di manifest

```yaml
containers:
- name: transfer
  image: studi-kasus/transfer   # tag dari overlay
  imagePullPolicy: IfNotPresent
  ports: [{ name: http, containerPort: 8082 }]
  envFrom:
  - configMapRef: { name: transfer }
  - secretRef:    { name: transfer }
  readinessProbe:
    httpGet: { path: /readyz, port: http }
    periodSeconds: 5
    timeoutSeconds: 3
    failureThreshold: 2
  livenessProbe:
    httpGet: { path: /healthz, port: http }
    periodSeconds: 10
    failureThreshold: 3
  resources:
    requests: { cpu: 50m, memory: 64Mi }
    limits:   { cpu: 500m, memory: 256Mi }
```

Sumber: `k8s/base/transfer/deployment.yaml` · *tag* `lab-09`

- Konsepnya dibahas di Hari 1 (anatomi service): `/healthz` menjawab “apakah proses ini macet dan perlu di-*restart*?”, `/readyz` menjawab “apakah *pod* ini boleh menerima *traffic* sekarang?”
- `readinessProbe` gagal 2 kali berturut-turut (±10 detik): *pod* dikeluarkan dari *endpoint* Service, tidak di-*restart*
- `livenessProbe` gagal 3 kali berturut-turut (±30 detik): kubelet me-*restart container*
- `startupProbe` untuk service yang *startup*-nya lama, misalnya memuat *cache* besar. Selama belum lolos, *liveness* dan *readiness* belum dijalankan. Studi kasus tidak menggunakannya karena service siap dalam waktu kurang dari satu detik
- *Health service* gRPC account tidak memeriksa database, sehingga tidak digunakan sebagai *readiness probe*

PROBE

Tindakan Kubernetes pada setiap skenario

Kondisi	`/healthz`	`/readyz`	Tindakan Kubernetes dan alasannya
Startup, database belum bisa dihubungi	Belum menjawab	Belum menjawab	*Ping* database saat *startup* gagal, sehingga proses berhenti dengan *error*. *Container* di-*restart* dengan jeda yang makin panjang (`CrashLoopBackOff`). *Pod* belum pernah menerima *traffic*
Database milik service mati setelah *startup*	200	503	*Pod* dikeluarkan dari *endpoint* Service tanpa *restart*, lalu menerima *traffic* lagi setelah *ping* berhasil
Proses macet, server HTTP tidak menjawab	*Timeout*	*Timeout*	*Container* di-*restart* setelah 3 kegagalan *liveness*, karena hanya *restart* yang bisa memperbaiki proses yang macet
Overload	Lambat	Lambat, atau 503 bila *ping* database melewati 2 detik	Jawaban yang melewati `timeoutSeconds` dihitung gagal (*readiness* 3 detik, *liveness* 1 detik). *Readiness* mengurangi *traffic* ke *pod* tersebut, tetapi *liveness* yang terlalu ketat bisa me-*restart pod* yang sebenarnya hanya sibuk
Account lambat atau mati (dilihat dari transfer)	200	200	Tidak ada tindakan. *Readiness* transfer tidak memeriksa account, karena gangguan account ditangani *circuit breaker* (Hari 2)

PERHATIAN

Kesalahan umum: *liveness* memeriksa database. Saat database mati, semua *pod* di-*restart* bersamaan, padahal *restart* tidak memperbaiki database.

Security context yang lolos SCC `restricted-v2`

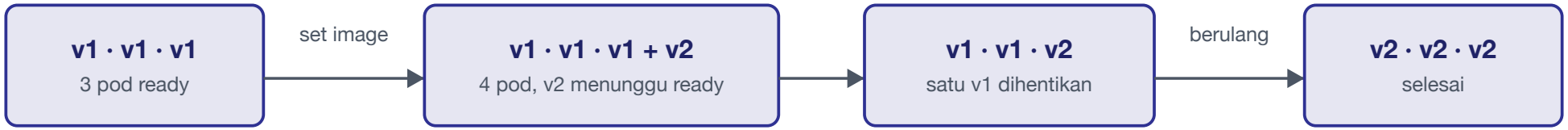
```yaml
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
# runAsUser sengaja tidak diisi
```

Sumber: `k8s/base/transfer/deployment.yaml` · *tag* `lab-09`

- Tidak ada `runAsUser`: platform yang memilih UID, sesuai perilaku OpenShift
- `readOnlyRootFilesystem` aman karena service tidak menulis *file*
- Manifest yang sama berlaku di k3d dan di *cluster* yang menerapkan kebijakan ketat

ROLLING UPDATE

Rolling update tanpa *downtime*




```
make images VERSION=1.0.1
make k3d-import VERSION=1.0.1
kubectl -n bank set image deployment/transfer \
  transfer=studi-kasus/transfer:1.0.1
kubectl -n bank rollout status deployment/transfer
kubectl -n bank rollout undo deployment/transfer
```

- Dengan `maxSurge: 1` dan `maxUnavailable: 0` di manifest, satu *pod* tambahan dibuat terlebih dahulu sehingga jumlah *pod* tidak pernah turun
- *Pod* lama baru dihentikan setelah *pod* baru lolos *readiness probe*. Bila *image* baru gagal *start*, *rollout* tertahan dan *pod* versi lama tetap melayani *request*
- Setiap perubahan mendapat `VERSION` baru. *Build* ulang dengan *tag* yang sama tidak mengubah *pod* yang berjalan, karena `IfNotPresent` dan referensi *image* yang tidak berubah tidak memicu *rollout*
- `rollout undo` mengembalikan versi sebelumnya. Peringatan `last-applied-configuration` yang muncul dapat diabaikan

`preStop` dan *grace period* saat *pod* dihentikan

```yaml
spec:
  # 5 s preStop + 60 s shutdownGrace
  # + 5 s flush telemetri, ditambah cadangan
  terminationGracePeriodSeconds: 75
  containers:
    - name: transfer
      lifecycle:
        preStop:
          sleep:
            seconds: 5
```

Sumber: `k8s/base/transfer/deployment.yaml` · *tag* `lab-09`

- *Pod* dikeluarkan dari daftar *endpoint* Service dan Ingress bersamaan dengan proses penghentiannya. Selama beberapa detik, *request* baru masih datang
- `preStop` `sleep` 5 detik menunda `SIGTERM` sampai *pod* tidak lagi menerima *request* baru. Aksi `sleep` bawaan (Kubernetes 1.30+) tidak membutuhkan *shell*, yang memang tidak ada di `scratch`
- `terminationGracePeriodSeconds` dihitung sejak `preStop` dimulai: `preStop` + *timeout shutdown* + *flush* telemetri 5 detik. Account dan notification 30 (5 + 10 + 5), transfer 75 (5 + 60 + 5, ditambah cadangan)
- Dengan nilai *default* 30 detik, Kubernetes akan mengirim `SIGKILL` ke transfer di tengah saga

Pod mengakses infrastruktur di *host*

- Hanya tiga service yang berjalan di k3d. PostgreSQL, Kafka, Hydra, dan `otel-lgtm` tetap berjalan dari `compose.yaml`.
- *Pod* mengakses *host* lewat nama `host.k3d.internal`.
- Kafka mengirim alamat *broker* ke klien (*advertised listener*). *Listener* tambahan pada *port* 19093 mengirim alamat `host.k3d.internal`, sementara klien di *host* tetap menggunakan `localhost:19092`.
- Token dari Hydra membawa `iss` = `http://localhost:14444/`. *Pod* tetap memeriksa *issuer* tersebut, tetapi mengambil JWKS dan token lewat `host.k3d.internal`.

PERHATIAN

Di *cluster* sungguhan, infrastruktur memiliki nama DNS sendiri sehingga perbedaan alamat *issuer* seperti ini tidak muncul. Bagian ini khusus untuk lab di laptop.

LAB 9

Menjalankan seluruh sistem di k3d

Tujuan

- Mengemas tiga service menjadi *image* minimal dan *non-root*
- Menerapkan manifest yang disediakan ke *cluster* k3d
- Melakukan *scaling*, perubahan konfigurasi, dan *rolling update*

Yang dikerjakan

- `make images VERSION=1.0.0`, `make k3d-up`, `make k3d-import VERSION=1.0.0`, `make k8s-apply`
- Transfer lewat Ingress `localhost:28080`, cek hasilnya di Grafana
- `kubectl scale` notification menjadi 3 dan 4 *replica*
- Hapus `KAFKA_GROUP_ID` dari ConfigMap notification, *apply*, lalu `rollout restart`; kembalikan nilainya dan `rollout restart` lagi
- `VERSION=1.0.1`, `set image`, `rollout status`, `rollout undo`

TIPS

Langkah lengkap ada di **Panduan Lab, bab Lab 9**. Titik awal: *tag* `lab-08`. Peserta yang tertinggal cukup melakukan `git checkout` ke *tag* tersebut.

CATATAN

`make k3d-up` menunggu 40–60 detik sampai DNS *cluster* mengenal `host.k3d.internal`. Bila *pod* masuk `CrashLoopBackOff` dengan pesan `no such host` setelah Docker atau laptop di-*restart*, jalankan `k3d cluster stop bank` lalu `k3d cluster start bank`.

05

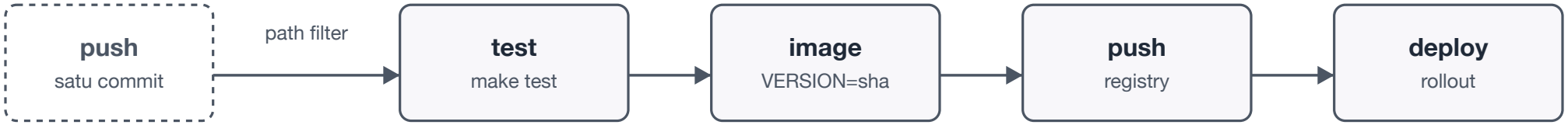
CI/CD per service

Pipeline per service, pemetaan ke Jenkins, dan GitOps sebagai konsep

MENGAPA PER SERVICE

Setiap service memiliki *pipeline* sendiri

- Sistem dipecah menjadi service terutama agar setiap service bisa di-*deploy* secara independen. Tujuan tersebut baru tercapai bila setiap service memiliki *pipeline* sendiri.
- Perubahan di `studi-kasus/transfer/` hanya menjalankan *pipeline* transfer. Perubahan di `proto/` menjalankan *pipeline* account dan transfer karena keduanya bergantung pada kontrak tersebut.
- Bila semua service berbagi satu *pipeline*, pola rilisnya kembali seperti monolith: semua service ikut di-*build* dan di-*deploy* setiap kali ada perubahan.



Workflow hanya memanggil target `make`

```yaml
on:
  push:
    branches: [main]
    paths:
      - studi-kasus/transfer/**
      - studi-kasus/proto/**
      - studi-kasus/go.work
      - studi-kasus/pipeline.mk
concurrency:
  group: pipeline-transfer
  cancel-in-progress: false
jobs:
  pipeline:
    runs-on: host
    env:
      REGISTRY: ${{ vars.REGISTRY }}
      IMAGE_OWNER: ${{ github.repository_owner }}
      NAMESPACE: ${{ vars.NAMESPACE }}
      VERSION: ${{ github.sha }}
    steps:
      - name: Checkout    # git fetch dengan token job
        run: ...
      - run: make -C studi-kasus/transfer test
      - run: make -C studi-kasus/transfer image
      - run: make -C studi-kasus/transfer push
      - run: make -C studi-kasus/transfer deploy
```

Sumber: `.forgejo/workflows/transfer.yaml` · *tag* `lab-10`

- Workflow tidak menggunakan `uses:`, sehingga setiap *step* berupa perintah *shell* biasa
- Logika *build* ada di `Makefile`, sehingga bisa dijalankan sama persis di laptop
- *Checkout* berupa `git fetch` dengan token yang diterbitkan runner untuk *job* tersebut. Isi lengkapnya ada di `.forgejo/workflows/transfer.yaml`
- *Tag image* = SHA *commit*, sehingga setiap *image* bisa dilacak ke kode sumbernya
- `concurrency` mengantrekan *run* berikutnya tanpa membatalkan yang sedang berjalan, agar dua *deploy* tidak berebut Deployment yang sama

Target `image`, `push`, `deploy`

```
IMAGE = $(REGISTRY)/$(IMAGE_OWNER)/$(SERVICE):$(VERSION)

image:
  $(call require,REGISTRY IMAGE_OWNER VERSION)
  docker build -f Dockerfile --label \
    org.opencontainers.image.revision=$(VERSION) \
    -t $(IMAGE) ..

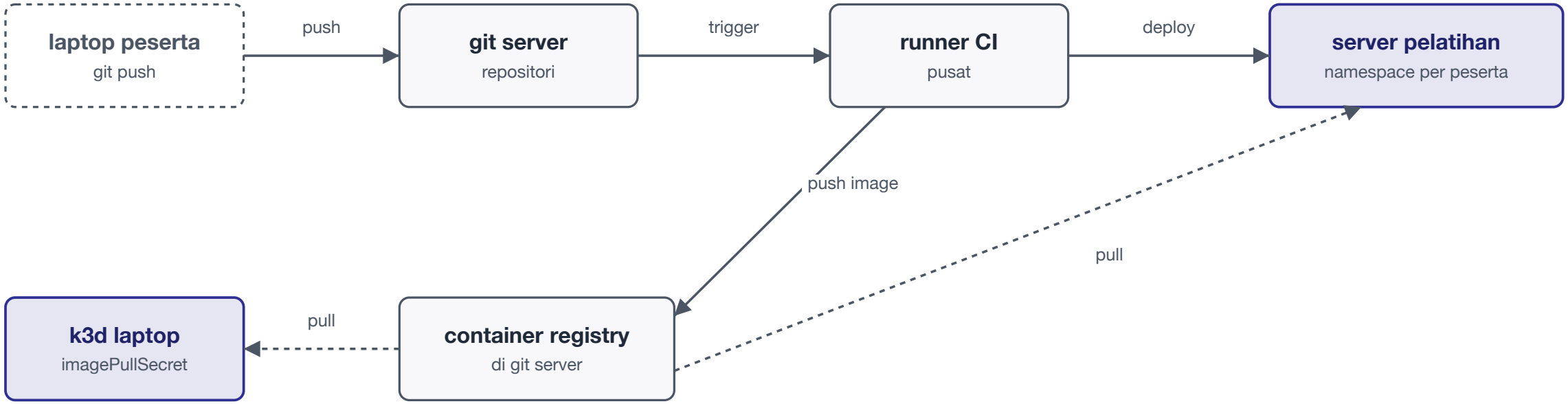
deploy:
  $(call require,REGISTRY IMAGE_OWNER VERSION NAMESPACE)
  @d=$$(kubectl -n $(NAMESPACE) get deployment/$(SERVICE) \
    --ignore-not-found -o name) || exit 1; \
  if [ -z "$$d" ]; then $(MAKE) apply || exit 1; fi
  kubectl -n $(NAMESPACE) set image \
    deployment/$(SERVICE) $(SERVICE)=$(IMAGE)
  kubectl -n $(NAMESPACE) annotate deployment/$(SERVICE) \
    kubernetes.io/change-cause="$(SERVICE) $(VERSION)" \
    --overwrite
  kubectl -n $(NAMESPACE) rollout status \
    deployment/$(SERVICE) --timeout=180s
```

Sumber: `pipeline.mk` · *tag* `lab-10`

- Target ini ada di `studi-kasus/pipeline.mk` dan di-*include* oleh `Makefile` setiap service
- `REGISTRY`, `IMAGE_OWNER`, `VERSION`, `NAMESPACE` wajib diisi. Bila ada yang kosong, target berhenti dengan pesan `missing required variables`
- *Tag* `latest` tidak digunakan agar *rollout* selalu menunjuk versi yang pasti
- *Deploy* pertama di *namespace* kosong menjalankan `apply`: ConfigMap, Service, Deployment (dan Ingress transfer) dibuat dari `k8s/overlays/server` dengan *image commit* ini
- `change-cause` diisi setelah `set image`, sehingga `rollout history` mencatat versi per revisi
- `deploy` menunggu `rollout status`, sehingga *pipeline* gagal bila *pod* baru tidak pernah *ready*

LAB 10

Arsitektur *pipeline* di lab



CATATAN

Alamat server dan akses setiap peserta dibagikan saat lab.

Pemetaan ke Bitbucket dan Jenkins *on-premise*

Lab	Jenkins
Repositori di git server	Repositori Bitbucket
Trigger `on: push`	*Webhook* Bitbucket ke Jenkins
`branches`, `paths:`	Hanya *Multibranch Pipeline* (*job* lain tidak mengisi `BRANCH_NAME`): `when { branch 'main' }` + `changeset`
`concurrency`	`disableConcurrentBuilds()`
Step `run: make test`	`stage('Test')`
Step `image`, `push`	`stage('Build Image')`, `stage('Push')`
Step `deploy`	`stage('Deploy')`
Variable dan *secret* runner	*Environment folder* dan Jenkins *credentials*

```
pipeline {
  agent any
  options { disableConcurrentBuilds() }
  environment { VERSION = "${env.GIT_COMMIT}" }
  stages { stage('transfer') {
    when { allOf { branch 'main'
      anyOf { triggeredBy 'UserIdCause'
              changeset 'studi-kasus/transfer/**'
              changeset 'studi-kasus/proto/**' } } }
    stages {
      stage('Test') { steps {
        script { if (!env.GIT_COMMIT ||
          env.GIT_COMMIT == 'null') error('...') }
        sh 'make -C studi-kasus/transfer test' } }
      stage('Build Image') { steps {
        sh 'make -C studi-kasus/transfer image' } }
      stage('Push') { steps {
        withCredentials([usernamePassword(...)]) {
          sh 'make -C studi-kasus/transfer push' } } }
      stage('Deploy') { steps {
        withCredentials([file(...)]) {
          sh 'make -C studi-kasus/transfer deploy' } } }
    } } }
}
```

Sumber: `transfer/Jenkinsfile` · *tag* `lab-10`

TIGA RUNNER

Satu *pipeline*, tiga runner CI

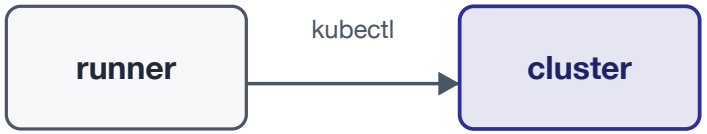
Tahap	Runner git server	Jenkins	GitHub Actions
Checkout	`git fetch`, *token job*	*Checkout* SCM bawaan	`git fetch`, `GITHUB_TOKEN`
Tools	Terpasang di host	Terpasang di *agent*	`install-tools.sh`, versi dan SHA-256 tetap
Test	`run: make test`	`stage('Test')`	`run: make test`
Build image	`run: make image`	`stage('Build Image')`	`run: make image`
Push	Registry git server	`stage('Push')`	`ghcr.io`, `GITHUB_TOKEN`
Deploy	`run: make deploy`	`stage('Deploy')`	`e2e-k3d.yaml`: `make pipeline-k3d` ke k3d di *runner*

CATATAN

Run GitHub Actions dapat dibuka publik di `github.com/artivisi/training-microservices-2026-01/actions`. `e2e-k3d` gagal bila ada respons selain 2xx selama *rolling update* transfer.

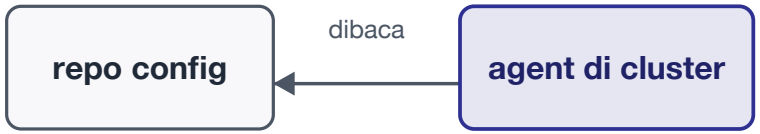
GitOps menyamakan *state cluster* dengan isi repositori

Model *push* (Lab 10)



- Runner menyimpan *credential* untuk mengakses *cluster*
- Perubahan manual di *cluster* tidak terdeteksi
- Sederhana dan cocok sebagai langkah awal

Model *pull* (GitOps)



- *Agent* GitOps di dalam *cluster* membandingkan isi repositori dengan *state cluster*, lalu menyamakannya
- *Rollback* cukup dengan `git revert`, dan riwayat perubahan tercatat sebagai riwayat *commit*
- CI hanya memperbarui *tag image* di repositori konfigurasi

CATATAN

GitOps dibahas sebagai arah pengembangan. Lab tidak memasang *tools* GitOps. Konsepnya cukup dipahami agar organisasi dapat menilainya ketika sudah siap.

LAB 10

Dari *commit* hingga *rolling update*

Tujuan

- Mengamati *pipeline* per service berjalan otomatis setelah *push*
- Melacak satu *commit* hingga menjadi *image* dengan *tag* SHA dan *pod* baru di *cluster*

Yang dikerjakan

- Membaca *workflow* `transfer.yaml` dan target `make` yang dipanggilnya
- Mengubah kode transfer, *commit*, lalu *push* ke git server
- Memantau *log pipeline*: *test*, *build image*, *push*, *deploy*. *Run* pertama di *namespace* kosong mencatat `first deploy, applying the overlay`
- Memeriksa `kubectl rollout history` (versi per revisi) dan versi *image* yang berjalan
- Membandingkan *workflow* tersebut dengan *Jenkinsfile* yang setara

TIPS

Langkah lengkap ada di **Panduan Lab, bab Lab 10**. Titik awal: *tag* `lab-09`. Peserta yang tertinggal cukup melakukan `git checkout` ke *tag* tersebut.

PERHATIAN

Bagian *deploy* ke k3d dengan runner lokal didemonstrasikan instruktur. Runner tidak dipasang di laptop peserta.

06

Operasional di *cloud*

HPA, penempatan *observability stack*, dan pemetaan ke OpenShift

SCALING

Horizontal Pod Autoscaler (HPA)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: transfer
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: transfer
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Ilustrasi, tidak diambil dari repositori studi kasus.

- HPA menambah atau mengurangi *replica* berdasarkan metrik, umumnya CPU terhadap `requests`
- Syarat: `resources.requests` terisi dan *metrics-server* berjalan (k3s menyediakannya)
- Service harus *stateless* agar *replica* bisa ditambah kapan saja
- *Scaling* transfer tidak menolong bila yang lambat adalah database account

Penempatan *observability stack*

Di dalam *cluster* aplikasi

- *Collector* OpenTelemetry sebagai DaemonSet atau *sidecar* di setiap *cluster*
- Penyimpanan jangka pendek dekat dengan sumber data
- Risiko: gangguan *cluster* ikut mematikan Grafana, Tempo, dan Loki yang dibutuhkan untuk menyelidiki gangguan tersebut

Cluster terpisah atau terpusat

- Tempo, Loki, Prometheus, dan Grafana di *cluster* khusus *observability*
- Satu tempat untuk melihat semua *environment*
- Data tetap tersedia saat *cluster* aplikasi bermasalah

- Pola yang umum: *collector* di setiap *cluster* aplikasi, penyimpanan dan Grafana di tempat terpusat.
- Kode aplikasi tidak perlu diubah karena service hanya mengenal `OTEL_EXPORTER_OTLP_ENDPOINT`. Cukup alamat tersebut yang diganti.
- *Log* dan *trace* bisa berisi data nasabah. Retensi dan hak akses mengikuti klasifikasi data yang sama dengan aplikasinya.

OPENSHIFT

Pemetaan Kubernetes ke OpenShift

Kubernetes (lab)	OpenShift	Catatan
Ingress (Traefik)	Route	OpenShift juga menerima Ingress dan mengubahnya menjadi Route. Route menambahkan opsi TLS *edge*, *passthrough*, *reencrypt*
`securityContext` + Pod Security	SCC `restricted-v2`	Pengganti SCC `restricted` sejak OpenShift 4.11. UID acak, tanpa *privilege escalation*, *capabilities* dibuang, *seccomp* `RuntimeDefault`. Manifest Lab 9 sudah memenuhinya
`kubectl`	`oc`	`oc` mencakup semua perintah `kubectl`, ditambah `oc login`, `oc new-project`, `oc adm`
Namespace	*Project*	*Project* = *namespace* dengan anotasi dan kuota tambahan
Registry eksternal	*Internal registry* + ImageStream	*Pipeline* tetap bisa melakukan *push image* ke *registry* eksternal

TIPS

Uji kecocokan sebelum ke OpenShift: jalankan *image* dengan UID acak dan *root filesystem read-only*. Bila keduanya lolos, sebagian besar masalah SCC sudah tertangani.

Route vs Ingress

```yaml
# Kubernetes: Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: transfer }
spec:
  rules:
  - http:
      paths:
      - path: /transfers
        pathType: Prefix
        backend:
          service:
            name: transfer
            port: { number: 8082 }
```

Sumber: `k8s/base/transfer/ingress.yaml` · *tag* `lab-09`

```yaml
# OpenShift: Route
apiVersion: route.openshift.io/v1
kind: Route
metadata: { name: transfer }
spec:
  path: /transfers
  to:
    kind: Service
    name: transfer
  port:
    targetPort: 8082
  tls:
    termination: edge
```

Ilustrasi, tidak diambil dari repositori studi kasus.

Kedua objek mengarahkan *traffic* HTTP ke Service yang sama. Perbedaannya hanya pada objek *routing*, sedangkan Deployment dan Service tetap sama.

RINGKASAN

Hasil empat hari

- Tiga service Go dengan *database-per-service* dalam satu repositori multi-modul.
- *Timeout*, *retry*, dan *circuit breaker* dengan failsafe-go, diuji dengan *fault injection*.
- Kontrak Protobuf, gRPC *server* dan *client* dengan *interceptor*.
- Saga dengan kompensasi, *transactional outbox*, dan *idempotent consumer* di Kafka.
- OAuth2 *client credentials* dari Hydra: *middleware* REST dan *interceptor* gRPC.
- `slog` dengan *correlation ID*, metrik Prometheus, *trace* OpenTelemetry, *dashboard* Grafana.
- Dockerfile *multi-stage* per service dan Docker Compose untuk seluruh sistem.
- Manifest Kubernetes untuk seluruh sistem, diuji di k3d.
- *Pipeline* CI/CD per service: *test*, *build image*, *deploy*.

Peta *concern* lengkap di studi kasus dan di *production*

#	*Concern*	Yang diterapkan di studi kasus	Yang ditambahkan di *production*
1	*Decomposition* dan kepemilikan data	Account, transfer, notification, masing-masing dengan database sendiri	Kepemilikan service per tim, katalog service dan kontrak API
2	Komunikasi *synchronous* dan *asynchronous*	REST untuk *channel*, gRPC transfer ke account, Kafka `transfer-events`	API *gateway*, versi kontrak yang terkelola, Kafka multi-*broker*
3	*Resiliency*	*Timeout*, *retry*, *circuit breaker* (failsafe-go), *probe*, *rolling update*	*Rate limiting* di *gateway*, uji *fault* berkala, *service mesh* bila kebijakan ingin diseragamkan di platform
4	Konsistensi data	Saga dengan kompensasi, *outbox*, *idempotent consumer*	Rekonsiliasi berkala, *monitoring* saga yang tertahan, *dead letter topic*
5	*Security*	OAuth2 *client credentials* dari Hydra, JWT di *middleware* dan *interceptor*	IdP organisasi, mTLS antar-service, rotasi kunci, API *gateway*
6	*Observability*	`slog` + *correlation ID*, Prometheus, OTel, `otel-lgtm`	*Stack* terpisah dengan retensi sesuai klasifikasi data, *alerting*, kontrol akses
7	*Configuration* (termasuk *centralized config* dan *secret*)	*Environment variable* tanpa *default*, ConfigMap dan Secret dari *overlay*	*Secret manager* + External Secrets Operator, enkripsi Secret di etcd
8	*Service discovery* dan *load balancing*	Service DNS `account:9091`, Ingress Traefik	*Load balancing* gRPC per *request* (*client-side* atau *service mesh*), Route OpenShift
9	*Packaging* dan *deployment*	Dockerfile *multi-stage* `scratch`, Deployment, *probe*, *rolling update* di k3d	Pemindaian dan penandatanganan *image*, PodDisruptionBudget, HPA
10	*Automation* (CI/CD)	*Pipeline* per service: *test*, *image*, *push*, *deploy* (model *push*)	Promosi antar-*environment* dengan persetujuan, GitOps

Platform Spring Cloud menyediakan *discovery* (Eureka), *load balancing*, *circuit breaker*, dan konfigurasi (Spring Cloud Config) sebagai *library* ekosistem Java. Platform Kubernetes menyediakan *discovery*, *load balancing*, konfigurasi, *restart*, dan *rollout* untuk service dalam bahasa apa pun. Service *standalone* di VM tidak memiliki keduanya, sehingga tim menyediakan sendiri setiap *concern* tersebut. Kebijakan *resiliency*, konsistensi data, pemeriksaan *security*, dan instrumentasi tetap di kode.

KEPUTUSAN DESAIN

Ringkasan keputusan desain empat hari

Hari	Keputusan	Pilihan	Konsekuensi yang diterima
H1	Bahasa service	Go	Tidak ada *framework* setara Spring Boot. Konfigurasi, *middleware*, dan telemetri dirangkai sendiri
H1	*Routing* HTTP	`ServeMux` standar (Go 1.22+)	Tanpa pengelompokan *route* serta *binding* dan validasi otomatis
H1	Akses database	pgx tanpa ORM	*Query* ditulis manual dan terikat ke PostgreSQL
H1	Kepemilikan data	Database per service	Tidak ada *join* dan transaksi lintas service, sehingga dibutuhkan saga
H2	*Resiliency*	failsafe-go	Dependensi tambahan dengan API *generic*. *Policy* ada di kode setiap service, perubahan parameter butuh *build* dan rilis ulang
H2	Panggilan internal	gRPC + protobuf	Tidak bisa dicoba langsung dengan curl, *load balancing* per koneksi, kontrak harus kompatibel
H2	API *channel*	REST	Dua gaya API dipelihara berdampingan
H3	Notifikasi	*Asynchronous messaging*	*Eventual consistency*, duplikat dan urutan harus ditangani
H3	*Broker*	Kafka	Jumlah *consumer* aktif per *group* dibatasi jumlah *partition*, *routing* lebih terbatas dibanding RabbitMQ
H3	Otorisasi antar-service	OAuth2 *client credentials*, Hydra	Token bergantung pada Hydra. Ganti IdP: ubah `OAUTH_*` URL dan sesuaikan *claim scope*
H3	*Observability* lab	`otel-lgtm`	Tidak untuk *production*. Komponen dipisah saat naik ke *production*
H4	Artefak rilis	*Container image* `scratch`, *non-root*	Membutuhkan *registry* dan pemindaian, tanpa *shell* untuk *debugging*
H4	Platform	Kubernetes	Kompleksitas operasional *cluster*
H4	*Cluster* training	k3d	k3s berbeda dengan OpenShift (SCC, Route)

KARTU REFERENSI

Pola Hari 4 beserta syarat penggunaannya

Pola	Masalah yang diselesaikan	Layak digunakan ketika
Multi-stage build + `scratch`	*Image* besar dan penuh paket yang tidak dibutuhkan	Service Go tanpa CGO
Readiness probe	*Traffic* masuk ke *pod* yang belum siap	Selalu, untuk setiap service yang menerima *traffic*
Rolling update	*Downtime* saat rilis	Service *stateless* dengan *readiness probe* yang benar
Pipeline per service	Service lain ikut dirilis setiap kali satu service berubah	Tim ingin merilis service secara independen
HPA	Beban berubah-ubah sepanjang hari	Service *stateless* yang *CPU-bound*
GitOps	*State cluster* menyimpang dari repositori	Banyak *cluster* dan kebutuhan audit perubahan yang ketat

EKSPLORASI MANDIRI

Langkah lanjutan setelah training

- Ulangi Lab 3 dan Lab 8 dengan nilai *fault* yang berbeda, lalu cocokkan perilaku *breaker* di *log*, metrik, dan *trace*.
- Tambahkan HPA untuk transfer di k3d, lalu beri beban dengan *load generator* sederhana dan amati jumlah *replica*.
- Jalankan *Jenkinsfile* salah satu service sebagai *Multibranch Pipeline* di Jenkins, lalu bandingkan *log* Jenkins dengan *log run* di runner lab.
- Jalankan *image* setiap service dengan UID acak dan `--read-only` untuk memastikan kecocokan dengan SCC `restricted-v2`.
- Pilih satu aplikasi internal, lalu tentukan pola mana saja di kartu referensi yang benar-benar dibutuhkan aplikasi tersebut.

Tanya jawab

Diskusi arsitektur, kendala penerapan, dan pertanyaan seputar materi empat hari